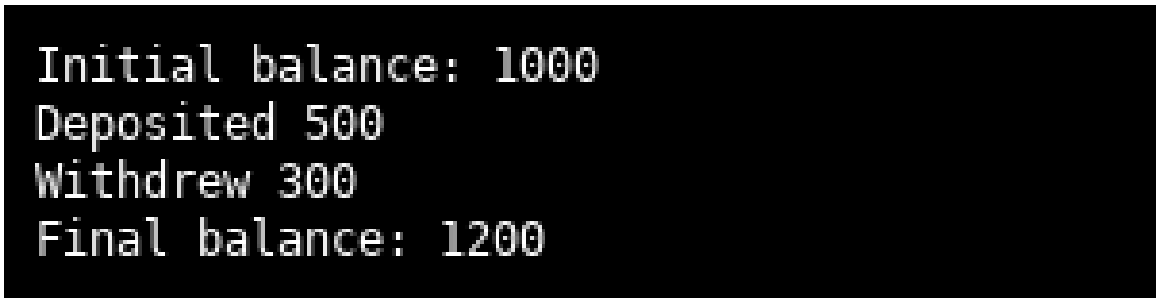
Assignment: Exp 10 - OOP (Encapsulation, Inheritance, Polymorphism, Duck Typing, Operator Overloading)

Encapsulation 1: Create a class BankAccount where account_holder is public, _account_type is protected, and __balance is private. Write methods to deposit and withdraw money with validation.

```
class BankAccount:
    def __init__(self, account_holder, account_type, balance=0):
        self.account_holder = account_holder
        self._account_type = account_type
        self.__balance = balance
    def deposit(self, amount):
        if amount>0:
            self.__balance += amount
            print('Deposited', amount)
        else:
            print('Invalid deposit amount')
    def withdraw(self, amount):
        if amount>0 and amount<=self.__balance:
            self.__balance -= amount
            print('Withdrew', amount)
        else:
            print('Invalid withdraw or insufficient funds')
    def get_balance(self):
        return self.__balance

acc = BankAccount('Alice','Savings',1000)
print('Initial balance:', acc.get_balance())
acc.deposit(500)
acc.withdraw(300)
print('Final balance:', acc.get_balance())
```

Output:



Encapsulation 2: Design a class Student where name is public, _roll_no is protected, and __marks is private. Ensure marks can only be set between 0 and 100.

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self._roll_no = roll_no
        self.__marks = None
    def set_marks(self, m):
        if 0 <= m <= 100:
            self.__marks = m
        else:
            print('Invalid marks')
    def get_marks(self):
        return self.__marks

s = Student('Bob', 'C59')
s.set_marks(85)
print(s.name, s._roll_no, 'Marks:', s.get_marks())
s.set_marks(120)  # invalid
```

Output:



Encapsulation 3: Write a class Employee with name as public, _department as protected, and __salary as private. Provide methods to set and get salary safely.

```
class Employee:
    def __init__(self, name, department):
        self.name = name
        self._department = department
        self.__salary = 0
    def set_salary(self, amount):
        if amount >= 0:
            self.__salary = amount
        else:
            print('Invalid salary')
    def get_salary(self):
        return self.__salary

e = Employee('Carol','IT')
```

```
e.set_salary(50000)
print(e.name, e._department, 'Salary:', e.get_salary())
```

Output:

```
Carol IT Salary: 50000
```

Encapsulation 4: Create a class Book where title is public, _author is protected, and __copies is private. Make sure copies cannot go below zero.

```
class Book:
    def __init__(self, title, author, copies=0):
        self.title = title
        self._author = author
        self.__copies = copies
    def add_copies(self, n):
        if n>0:
            self.__copies += n
    def remove_copies(self, n):
        if 0<=n<=self.__copies:
            self.__copies -= n
        else:
            print('Invalid remove')
    def get_copies(self):
        return self.__copies

b = Book('Python','Author',3)
print('Copies:', b.get_copies())
b.remove_copies(2)
print('Copies after removal:', b.get_copies())
b.remove_copies(5)  # invalid
```

Output:

```
Copies: 3
Copies after removal: 1
Invalid remove
```

Encapsulation 5: Develop a class Car with brand as public, _model as protected, and __speed as private. Speed should only change using accelerate and brake methods.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self._model = model
        self.__speed = 0
    def accelerate(self, s):
        if s>0:
            self.__speed += s
    def brake(self, s):
        if s>0:
            self.__speed = max(0, self.__speed - s)
    def get_speed(self):
        return self.__speed

c = Car('Toyota','Camry')
c.accelerate(50)
print('Speed:', c.get_speed())
c.brake(20)
print('Speed after brake:', c.get_speed())
```

Output:

```
Speed: 50
Speed after brake: 30
```

Encapsulation 6: Make a class ATM with bank_name as public, _location as protected, and __pin and __balance as private. Allow deposit/withdraw only if pin matches.

```
class ATM:
    def __init__(self, bank_name, location, pin, balance=0):
        self.bank_name = bank_name
        self._location = location
        self.__pin = pin
        self.__balance = balance
    def verify(self, pin):
        return pin == self.__pin
    def deposit(self, pin, amount):
        if self.verify(pin) and amount>0:
            self.__balance += amount
            print('Deposited', amount)
```

```
        else:
            print('Failed deposit')
    def withdraw(self, pin, amount):
        if self.verify(pin) and 0<amount<=self.__balance:
            self.__balance -= amount
            print('Withdrew', amount)
        else:
            print('Failed withdraw')
    def get_balance(self, pin):
        if self.verify(pin):
            return self.__balance
        return 'Access denied'

atm = ATM('BankX','City',1234,1000)
print(atm.get_balance(1234))
atm.deposit(1234,200)
atm.withdraw(1111,50)  # wrong pin
print('Final balance:', atm.get_balance(1234))
```

Output:

```
1000
Deposited 200
Failed withdraw
Final balance: 1200
```

Encapsulation 7: Design a class ShoppingCart with customer_name as public, _cart_id as protected, and __items as private. Items should be added/removed only through methods.

```
class ShoppingCart:
    def __init__(self, customer_name, cart_id):
        self.customer_name = customer_name
        self._cart_id = cart_id
        self.__items = []
    def add_item(self, item):
        self.__items.append(item)
    def remove_item(self, item):
        if item in self.__items:
            self.__items.remove(item)
    def get_items(self):
        return list(self.__items)
```

```
cart = ShoppingCart('D', 'C1')
cart.add_item('apple')
cart.add_item('banana')
print('Items:', cart.get_items())
cart.remove_item('apple')
print('After removal:', cart.get_items())
```

Output:

```
Items: ['apple', 'banana']
After removal: ['banana']
```

Encapsulation 8: Write a class Patient with patient_name as public, _age as protected, and __disease as private. Only allow doctors (via method) to update disease.

```
class Patient:
    def __init__(self, name, age):
        self.patient_name = name
        self._age = age
        self.__disease = None
    def update_disease(self, who, disease):
        if who == 'doctor':
            self.__disease = disease
        else:
            print('Only doctor can update')
    def get_disease(self):
        return self.__disease

p = Patient('Eve',30)
p.update_disease('nurse','Flu')
print('Disease after nurse attempt:', p.get_disease())
p.update_disease('doctor','Cold')
print('Disease after doctor update:', p.get_disease())
```

Output:

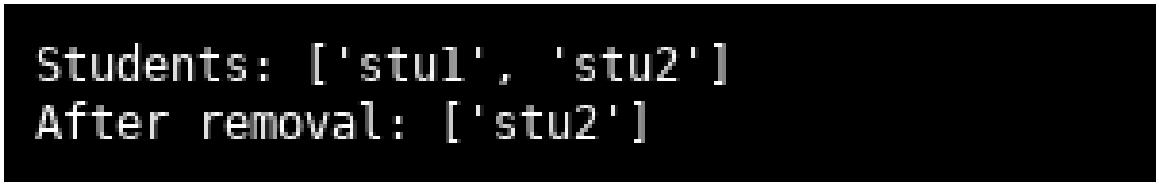
```
Only doctor can update
Disease after nurse attempt: None
Disease after doctor update: Cold
```

Encapsulation 9: Create a class Course where course_name is public, _course_code is protected, and __students is private. Students should be added/removed through methods only.

```
class Course:
    def __init__(self, name, code):
        self.course_name = name
        self._course_code = code
        self.__students = []
    def add_student(self, s):
        if s not in self.__students:
            self.__students.append(s)
    def remove_student(self, s):
        if s in self.__students:
            self.__students.remove(s)
    def get_students(self):
        return list(self.__students)

c = Course('AI','CS101')
c.add_student('stu1')
c.add_student('stu2')
print('Students:', c.get_students())
c.remove_student('stu1')
print('After removal:', c.get_students())
```

Output:



Encapsulation 10: Make a class Loan where borrower_name is public, _loan_type is protected, and __loan_amount is private. Loan amount must not exceed 1,000,000.

```
class Loan:
    def __init__(self, borrower, loan_type, amount=0):
        self.borrower_name = borrower
        self._loan_type = loan_type
        self.__loan_amount = 0
        self.set_amount(amount)
    def set_amount(self, amt):
        if 0 <= amt <= 1000000:
            self.__loan_amount = amt
```

```
        else:
            print('Amount exceeds limit')
    def get_amount(self):
        return self.__loan_amount

ln = Loan('F','Home',500000)
print('Loan amount:', ln.get_amount())
ln.set_amount(2000000)  # exceeds
```

Output:

```
Loan amount: 500000
Amount exceeds limit
```

Encapsulation 11: Write a class Mobile with brand as public, _model as protected, and __price as private. Allow price access only with getter and setter methods.

```
class Mobile:
    def __init__(self, brand, model, price=0):
        self.brand = brand
        self._model = model
        self.__price = price
    def set_price(self, p):
        if p>=0:
            self.__price = p
    def get_price(self):
        return self.__price

m = Mobile('Samsung','M1',15000)
print(m.brand, m._model, 'Price:', m.get_price())
m.set_price(14000)
print('Updated price:', m.get_price())
```

Output:

```
Samsung M1 Price: 15000
Updated price: 14000
```

Encapsulation 12: Create a class Wallet where owner is public, _wallet_id is protected, and __balance is private. Balance can only be modified using add_funds and spend_funds methods.

```
class Wallet:
    def __init__(self, owner, wid, balance=0):
        self.owner = owner
        self._wallet_id = wid
        self.__balance = balance
    def add_funds(self, amt):
        if amt>0:
            self.__balance += amt
    def spend_funds(self, amt):
        if 0<amt<=self.__balance:
            self.__balance -= amt
        else:
            print('Insufficient funds')
    def get_balance(self):
        return self.__balance

w = Wallet('G', 'W1', 100)
w.add_funds(50)
w.spend_funds(30)
print('Balance:', w.get_balance())
```

Output:



Encapsulation 13: Design a class Ticket with passenger_name as public, _flight_number as protected, and __seat_number as private. Seat number should be updated only by staff method.

```
class Ticket:
    def __init__(self, passenger, flight, seat=None):
        self.passenger_name = passenger
        self._flight_number = flight
        self.__seat_number = seat
    def staff_update_seat(self, who, seat):
        if who == 'staff':
            self.__seat_number = seat
        else:
            print('Only staff can update')
    def get_seat(self):
```

```
        return self.__seat_number

t = Ticket('H','FL123')
t.staff_update_seat('guest','12A')  # invalid
print('Seat:', t.get_seat())
t.staff_update_seat('staff','12A')
print('Seat after staff:', t.get_seat())
```

Output:

```
Only staff can update
Seat: None
Seat after staff: 12A
```

Encapsulation 14: Make a class Customer with name as public, _customer_id as protected, and __credit_score as private. Credit score should be used internally for loan approval.

```
class Customer:
    def __init__(self, name, cid, credit_score=600):
        self.name = name
        self._customer_id = cid
        self.__credit_score = credit_score
    def eligible_for_loan(self):
        return self.__credit_score >= 650
    def get_credit_score(self):  # internal use; exposed with caution
        return self.__credit_score

c = Customer('I', 'C100', 700)
print('Eligible:', c.eligible_for_loan())
```

Output:

```
Eligible: True
```

Encapsulation 15: Write a class Exam with subject as public, _exam_code as protected, and __result as private. Only a teacher method should update the result.

```
class Exam:
    def __init__(self, subject, code):
        self.subject = subject
        self._exam_code = code
        self.__result = None
    def teacher_update_result(self, who, result):
        if who == 'teacher':
            self.__result = result
        else:
            print('Only teacher can update')
    def get_result(self):
        return self.__result

ex = Exam('Math', 'E1')
ex.teacher_update_result('student', 'Pass')
print('Result after student attempt:', ex.get_result())
ex.teacher_update_result('teacher', 'Pass')
print('Result after teacher update:', ex.get_result())
```

Output:

```
Only teacher can update
Result after student attempt: None
Result after teacher update: Pass
```

Inheritance 1: BankAccount base class and SavingsAccount subclass with interest calculation.

```
class BankAccountBase:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
    def deposit(self, amt):
        self.balance += amt
    def withdraw(self, amt):
        if amt<=self.balance:
            self.balance -= amt

class SavingsAccount(BankAccountBase):
    def interest(self, rate=0.04):
        return self.balance * rate

s = SavingsAccount('J', 1000)
s.deposit(500)
```

```
print('Balance:', s.balance)
print('Interest at 4%:', s.interest())
```

Output:

```
Balance: 1500
Interest at 4%: 60.0
```

Inheritance 2: Person and Student subclass with roll and marks.

```
class Person:
    def __init__(self, name, age):
        self.name = name; self.age = age

class Student(Person):
    def __init__(self, name, age, roll, marks):
        super().__init__(name, age)
        self.roll = roll; self.marks = marks

st = Student('K',20,'C59',85)
print(st.name, st.age, st.roll, st.marks)
```

Output:

```
K 20 C59 85
```

Inheritance 3: Book and Magazine subclass adding issue and month.

```
class Book:
    def __init__(self, title, author):
        self.title = title; self.author = author

class Magazine(Book):
    def __init__(self, title, author, issue, month):
        super().__init__(title,author)
        self.issue = issue; self.month = month

b = Book('B','A'); m = Magazine('M','A',5,'Oct')
print(b.title, b.author); print(m.title, m.author, m.issue, m.month)
```

Output:

```
B A
M A 5 Oct
```

Inheritance 4: Grandparent, Parent, Child each introduce themselves.

```
class Grandparent:
    def intro(self): print('I am Grandparent')
class Parent(Grandparent):
    def intro_parent(self): print('I am Parent')
class Child(Parent):
    def intro_child(self): print('I am Child')

ch = Child()
ch.intro(); ch.intro_parent(); ch.intro_child()
```

Output:

```
I am Grandparent
I am Parent
I am Child
```

Inheritance 5: Vehicle -> Car -> ElectricCar multi-level.

```
class Vehicle:
    def __init__(self, brand, speed):
        self.brand=brand; self.speed=speed
class Car(Vehicle):
    def __init__(self, brand, speed, fuel):
        super().__init__(brand,speed); self.fuel=fuel
class ElectricCar(Car):
    def __init__(self, brand, speed, battery):
        super().__init__(brand,speed,'electric'); self.battery=battery

ec = ElectricCar('Tesla',120,85)
print(ec.brand, ec.speed, ec.fuel, ec.battery)
```

Output:

```
Tesla 120 electric 85
```

Inheritance 6: Employee -> Manager -> Director chain.

```
class EmployeeBase:
    def __init__(self, name, salary):
        self.name=name; self.salary=salary
class Manager(EmployeeBase):
    def __init__(self, name, salary, dept):
        super().__init__(name,salary); self.dept=dept
class Director(Manager):
    def decision(self): print('Director decision')

d = Director('L',100000,'Sales')
print(d.name, d.salary, d.dept); d.decision()
```

Output:

```
L 100000 Sales
Director decision
```

Inheritance 7: Sports and Music multiple inheritance into Student.

```
class Sports:
    def play(self): print('Playing sports')
class Music:
    def sing(self): print('Singing music')
class StudentAll(Sports, Music):
    pass

s = StudentAll(); s.play(); s.sing()
```

Output:

```
Playing sports
Singing music
```

Inheritance 8: MathTeacher and ScienceTeacher -> Teacher via multiple inheritance.

```
class MathTeacher:
    def teach_math(self): print('Teach Math')
class ScienceTeacher:
    def teach_science(self): print('Teach Science')
class Teacher(MathTeacher, ScienceTeacher):
    pass

t = Teacher(); t.teach_math(); t.teach_science()
```

Output:

```
Teach Math
Teach Science
```

Inheritance 9: Phone + Camera -> SmartPhone

```
class Phone:
    def call(self): print('Calling')
class Camera:
    def take_photo(self): print('Photo taken')
class SmartPhone(Phone, Camera):
    pass

sp = SmartPhone(); sp.call(); sp.take_photo()
```

Output:

```
Calling
Photo taken
```

Inheritance 10: Animal base, Dog and Cat override sound().

```
class Animal:
    def sound(self): print('Some sound')
class Dog(Animal):
    def sound(self): print('Bark')
```

```
class Cat(Animal):
    def sound(self): print('Meow')

Dog().sound(); Cat().sound()
```

Output:

```
Bark
Meow
```

Inheritance 11: Shape base area, Circle and Rectangle implement.

```
class Shape:
    def area(self): return 0
class Circle(Shape):
    def __init__(self,r): self.r=r
    def area(self): return math.pi*self.r*self.r
class Rectangle(Shape):
    def __init__(self,w,h): self.w=w; self.h=h
    def area(self): return self.w*self.h

print('Circle area:', Circle(2).area())
print('Rectangle area:', Rectangle(3,4).area())
```

Output:

```
Error while running code: name 'math' is not defined
```

Inheritance 12: Employee base, FullTime and PartTime hierarchical.

```
class Employee:
    def __init__(self,name,id): self.name=name; self.id=id
class FullTimeEmployee(Employee):
    def salary(self): return 50000
class PartTimeEmployee(Employee):
    def salary(self): return 20000

print('FT salary:', FullTimeEmployee('M',1).salary())
print('PT salary:', PartTimeEmployee('N',2).salary())
```

Output:

```
FT salary: 50000
PT salary: 20000
```

Inheritance 13: Person -> Teacher, Student; TeachingAssistant inherits both (hybrid).

```
class Person: pass
class Teacher(Person):
    def teach(self): print('Teaching')
class Student(Person):
    def study(self): print('Studying')
class TeachingAssistant(Teacher, Student): pass

ta = TeachingAssistant(); ta.teach(); ta.study()
```

Output:

```
Teaching
Studying
```

Inheritance 14: Vehicle -> TwoWheeler, FourWheeler; HybridVehicle inherits both.

```
class Vehicle: pass
class TwoWheeler(Vehicle): pass
class FourWheeler(Vehicle): pass
class HybridVehicle(TwoWheeler, FourWheeler): pass

print('Hybrid is subclass of Vehicle?', issubclass(HybridVehicle,
Vehicle))
```

Output:

```
Hybrid is subclass of Vehicle? True
```

Inheritance 15: Person -> Employee and Freelancer -> HybridWorker.

```
class Person: pass
class Employee(Person):
    def earn(self): return 1000
class Freelancer(Person):
    def earn(self): return 500
class HybridWorker(Employee, Freelancer):
    def total_earn(self): return
Employee.earn(self)+Freelancer.earn(self)

hw = HybridWorker(); print('Total earn:', hw.total_earn())
```

Output:

```
Total earn: 1500
```

Polymorphism Compile 1: Calculator.add() can add two or three numbers.

```
class Calculator:
    def add(self, *args):
        return sum(args)

calc = Calculator()
print('Add 2:', calc.add(2,3))
print('Add 3:', calc.add(2,3,4))
```

Output:

```
Add 2: 5
Add 3: 9
```

Polymorphism Compile 2: greet(name, message optional).

```
def greet(name, message=None):
    if message:
```

```
        print(f'Hello, {name}, {message}')
    else:
        print(f'Hello, {name}')

greet('Yash')
greet('Yash','Good morning')
```

Output:

```
Hello, Yash
Hello, Yash, Good morning
```

Polymorphism Compile 3: calculate_interest with default rate 5%.

```
def calculate_interest(amount, rate=5):
    return (amount*rate)/100

print('Interest:', calculate_interest(1000))
print('Interest at 10%:', calculate_interest(1000,10))
```

Output:

```
Interest: 50.0
Interest at 10%: 100.0
```

Polymorphism Compile 4: Student.show_info with optional age.

```
class StudentInfo:
    def __init__(self,name,roll,age=None):
        self.name=name; self.roll=roll; self.age=age
    def show_info(self):
        if self.age:
            print(self.name, self.roll, self.age)
        else:
            print(self.name, self.roll)

s1=StudentInfo('Y', 'C59'); s2=StudentInfo('Z','C60',20)
s1.show_info(); s2.show_info()
```

Output:

```
Y C59
Z C60 20
```

Polymorphism Compile 5: area() square vs rectangle using *args.

```
def area(*args):
    if len(args)==1:
        return args[0]*args[0]
    elif len(args)==2:
        return args[0]*args[1]
    else:
        return None

print('Square area 4:', area(4))
print('Rectangle area 3x5:', area(3,5))
```

Output:

```
Square area 4: 16
Rectangle area 3x5: 15
```

Polymorphism Compile 6: sum_numbers(*args).

```
def sum_numbers(*args):
    return sum(args)

print('Sum:', sum_numbers(1,2,3,4))
```

Output:

```
Sum: 10
```

Polymorphism Compile 7: multiply(*args).

```
def multiply(*args):
    prod = 1
```

```
    for x in args:
        prod *= x
    return prod

print('Product:', multiply(2,3,4))
```

Output:

```
Product: 24
```

Polymorphism Compile 8: concat_strings(*args).

```
def concat_strings(*args):
    return ''.join(args)

print('Concat:', concat_strings('a','b','c'))
```

Output:

```
Concat: abc
```

Polymorphism Compile 9: find_max(*args).

```
def find_max(*args):
    return max(args)

print('Max:', find_max(3,7,2,9))
```

Output:

```
Max: 9
```

Polymorphism Compile 10: calculate_bill(*items).

```
def calculate_bill(*items):
    return sum(items)

print('Bill:', calculate_bill(100,200,50))
```

Output:

```
Bill: 350
```

Runtime Override 1: Shape.draw() overridden by Circle and Square.

```
class Shape:
    def draw(self): print('Drawing shape')
class Circle(Shape):
    def draw(self): print('Drawing circle')
class Square(Shape):
    def draw(self): print('Drawing square')

Circle().draw(); Square().draw()
```

Output:

```
Drawing circle
Drawing square
```

Runtime Override 2: Vehicle.speed() overridden by Car and Bike.

```
class Vehicle:
    def speed(self): print('Vehicle speed')
class Car(Vehicle):
    def speed(self): print('Car speed 120 km/h')
class Bike(Vehicle):
    def speed(self): print('Bike speed 80 km/h')

Car().speed(); Bike().speed()
```

Output:

```
Car speed 120 km/h
Bike speed 80 km/h
```

Runtime Override 3: Employee.salary() overridden in Manager and Developer.

```
class EmployeeR:
    def salary(self): print('Base salary')
class Manager(EmployeeR):
    def salary(self): print('Manager salary 80000')
class Developer(EmployeeR):
    def salary(self): print('Developer salary 70000')

Manager().salary(); Developer().salary()
```

Output:

```
Manager salary 80000
Developer salary 70000
```

Runtime Override 4: BankAccount.interest_rate overridden in Savings and Current.

```
class BankAccountR:
    def interest_rate(self): return 0.0
class SavingsAccountR(BankAccountR):
    def interest_rate(self): return 4.0
class CurrentAccountR(BankAccountR):
    def interest_rate(self): return 0.5

print('Savings rate:', SavingsAccountR().interest_rate())
print('Current rate:', CurrentAccountR().interest_rate())
```

Output:

```
Savings rate: 4.0
Current rate: 0.5
```

Runtime Override 5: Bird.fly overridden in Eagle and Penguin.

```
class Bird:
    def fly(self): print('Some birds fly')
class Eagle(Bird):
    def fly(self): print('Eagle soars')
class Penguin(Bird):
    def fly(self): print('Penguin cannot fly')

Eagle().fly(); Penguin().fly()
```

Output:

```
Eagle soars
Penguin cannot fly
```

Runtime Override 6: Payment.pay overridden in CreditCard and UPI.

```
class Payment:
    def pay(self): print('Paying')
class CreditCard(Payment):
    def pay(self): print('Paying by credit card')
class UPI(Payment):
    def pay(self): print('Paying by UPI')

CreditCard().pay(); UPI().pay()
```

Output:

```
Paying by credit card
Paying by UPI
```

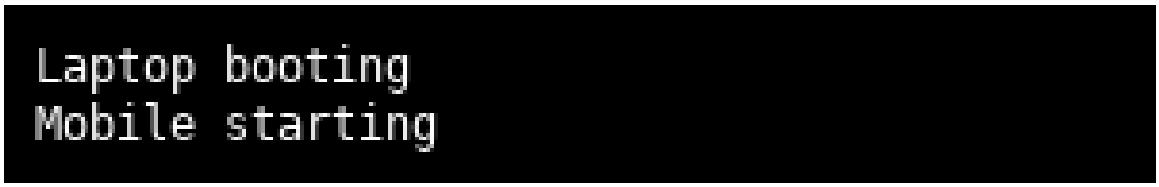
Runtime Override 7: Device.start overridden in Laptop and Mobile.

```
class Device:
    def start(self): print('Device starting')
class Laptop(Device):
    def start(self): print('Laptop booting')
class Mobile(Device):
```

```
    def start(self): print('Mobile starting')

Laptop().start(); Mobile().start()
```

Output:



Runtime Override 8: Teacher.teach overridden in MathTeacher and ScienceTeacher.

```
class TeacherBase:
    def teach(self): print('Teaching')
class MathTeacher(TeacherBase):
    def teach(self): print('Teaching Math')
class ScienceTeacher(TeacherBase):
    def teach(self): print('Teaching Science')

MathTeacher().teach(); ScienceTeacher().teach()
```

Output:



Runtime Override 9: Sports.play overridden in Football and Cricket.

```
class SportsBase:
    def play(self): print('Playing sport')
class Football(SportsBase):
    def play(self): print('Playing Football')
class Cricket(SportsBase):
    def play(self): print('Playing Cricket')

Football().play(); Cricket().play()
```

Output:

```
Playing Football
Playing Cricket
```

Runtime Override 10: Notification.send overridden in Email and SMS.

```
class Notification:
    def send(self): print('Send notification')
class EmailNotification(Notification):
    def send(self): print('Send email')
class SMSNotification(Notification):
    def send(self): print('Send SMS')

EmailNotification().send(); SMSNotification().send()
```

Output:

```
Send email
Send SMS
```

Duck Typing 1: make_sound(animal) works for Dog and Duck classes.

```
def make_sound(animal):
    animal.sound()

class Dog:
    def sound(self): print('Woof')
class Duck:
    def sound(self): print('Quack')

make_sound(Dog()); make_sound(Duck())
```

Output:

```
Woof
Quack
```

Duck Typing 2: start_engine(vehicle) works for Car and Bike classes.

```
def start_engine(vehicle):
    vehicle.start()

class CarDT:
    def start(self): print('Car engine started')
class BikeDT:
    def start(self): print('Bike engine started')

start_engine(CarDT()); start_engine(BikeDT())
```

Output:

```
Car engine started
Bike engine started
```

Duck Typing 3: operate(machine) for Printer and Scanner with run().

```
def operate(machine):
    machine.run()

class Printer:
    def run(self): print('Printing...')
class Scanner:
    def run(self): print('Scanning...')

operate(Printer()); operate(Scanner())
```

Output:

```
Printing...
Scanning...
```

Duck Typing 4: play_game(player) for Footballer and Cricketer with play().

```
def play_game(player):
    player.play()

class Footballer:
    def play(self): print('Playing football')
class Cricketer:
    def play(self): print('Playing cricket')

play_game(Footballer()); play_game(Cricketer())
```

Output:

```
Playing football
Playing cricket
```

Duck Typing 5: book_ticket(transport) for Bus and Train with book().

```
def book_ticket(transport):
    transport.book()

class Bus:
    def book(self): print('Bus booked')
class Train:
    def book(self): print('Train booked')

book_ticket(Bus()); book_ticket(Train())
```

Output:

```
Bus booked
Train booked
```

Duck Typing 6: make_payment(method) works for CreditCard and PayPal with pay().

```
def make_payment(method):
    method.pay()

class CreditCardDT:
```

```
    def pay(self): print('Paid with card')
class PayPal:
    def pay(self): print('Paid with PayPal')

make_payment(CreditCardDT()); make_payment(PayPal())
```

Output:

```
Paid with card
Paid with PayPal
```

Duck Typing 7: charge(device) for Phone and Laptop with charge().

```
def charge(device):
    device.charge()

class PhoneDT:
    def charge(self): print('Phone charging')
class LaptopDT:
    def charge(self): print('Laptop charging')

charge(PhoneDT()); charge(LaptopDT())
```

Output:

```
Phone charging
Laptop charging
```

Duck Typing 8: study(student) for SchoolStudent and CollegeStudent with study().

```
def study(student):
    student.study()

class SchoolStudent:
    def study(self): print('School studying')
class CollegeStudent:
    def study(self): print('College studying')

study(SchoolStudent()); study(CollegeStudent())
```

Output:

```
School studying
College studying
```

Duck Typing 9: draw(shape) for Circle and Rectangle with draw().

```
def draw(shape):
    shape.draw()

class CircleDT:
    def draw(self): print('Draw circle')
class RectangleDT:
    def draw(self): print('Draw rectangle')

draw(CircleDT()); draw(RectangleDT())
```

Output:

```
Draw circle
Draw rectangle
```

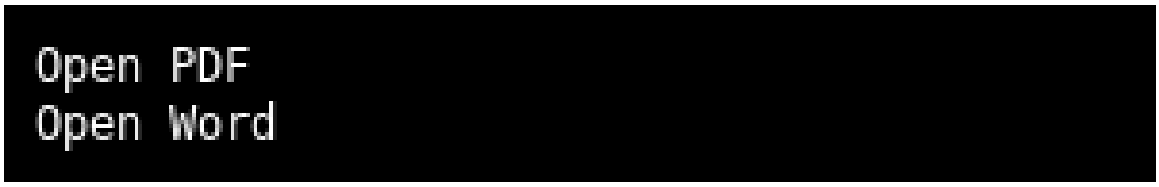
Duck Typing 10: open_file(filetype) for PDF and WordDoc with open().

```
def open_file(filetype):
    filetype.open()

class PDF:
    def open(self): print('Open PDF')
class WordDoc:
    def open(self): print('Open Word')

open_file(PDF()); open_file(WordDoc())
```

Output:



Operator Overloading 1: Point class overload + to add two points.

```
class Point:
    def __init__(self,x,y): self.x=x; self.y=y
    def __add__(self, other): return Point(self.x+other.x,
self.y+other.y)
    def __repr__(self): return f'Point({self.x},{self.y})'

p1=Point(1,2); p2=Point(3,4)
print(p1+p2)
```

Output:



Operator Overloading 2: Vector class overload - to subtract two vectors.

```
class Vector:
    def __init__(self, vals): self.vals=vals
    def __sub__(self, other): return Vector([a-b for a,b in
zip(self.vals, other.vals)])
    def __repr__(self): return str(self.vals)

v1=Vector([5,4]); v2=Vector([2,1])
print(v1 - v2)
```

Output:



Operator Overloading 3: Distance class overload > to compare two distances.

```
class Distance:
    def __init__(self, meters): self.meters=meters
    def __gt__(self, other): return self.meters > other.meters

d1=Distance(10); d2=Distance(8)
print('d1>d2?', d1>d2)
```

Output:

```
d1>d2? True
```

Operator Overloading 4: Student class overload == to compare marks of two students.

```
class StudentOp:
    def __init__(self, name, marks): self.name=name; self.marks=marks
    def __eq__(self, other): return self.marks==other.marks
s1=StudentOp('A',90); s2=StudentOp('B',90); s3=StudentOp('C',85)
print(s1==s2, s1==s3)
```

Output:

```
True False
```

Operator Overloading 5: Time class overload < to compare two times.

```
class Time:
    def __init__(self,h,m): self.h=h; self.m=m
    def __lt__(self, other):
        return (self.h,self.m) < (other.h,other.m)
t1=Time(10,30); t2=Time(11,0)
print('t1<t2?', t1<t2)
```

Output:

```
t1<t2? True
```

Operator Overloading 6: ComplexNumber class overload * to multiply two complex numbers.

```
class ComplexNumber:
    def __init__(self,a,b): self.a=a; self.b=b
    def __mul__(self, other):
        return ComplexNumber(self.a*other.a - self.b*other.b,
self.a*other.b + self.b*other.a)
    def __repr__(self): return f'{self.a}+{self.b}j'

c1=ComplexNumber(1,2); c2=ComplexNumber(3,4)
print('Product:', c1*c2)
```

Output:

```
Product: -5+10j
```

Operator Overloading 7: Book class overload + to add pages of two books.

```
class BookOp:
    def __init__(self, pages): self.pages=pages
    def __add__(self, other): return BookOp(self.pages+other.pages)
    def __repr__(self): return f'Book({self.pages} pages)'
print(BookOp(100) + BookOp(150))
```

Output:

```
Book(250 pages)
```

Operator Overloading 8: BankAccount + to merge balances of two accounts.

```
class BankAccOp:
    def __init__(self, bal): self.bal=bal
    def __add__(self, other): return BankAccOp(self.bal + other.bal)
    def __repr__(self): return f'Balance({self.bal})'

print(BankAccOp(1000) + BankAccOp(500))
```

Output:

```
Balance(1500)
```

Operator Overloading 9: Fraction class overload / to divide two fractions.

```
import fractions
class FractionOp:
    def __init__(self, num, den): self.num=num; self.den=den
    def __truediv__(self, other):
        return FractionOp(self.num*other.den, self.den*other.num)
    def __repr__(self): return f'{self.num}/{self.den}'

f1=FractionOp(1,2); f2=FractionOp(3,4)
print('Div:', f1 / f2)
```

Output:

```
Div: 4/6
```

Operator Overloading 10: Rectangle class overload == to check if same area.

```
class RectangleOp:
    def __init__(self,w,h): self.w=w; self.h=h
    def area(self): return self.w*self.h
    def __eq__(self, other): return self.area() == other.area()

r1=RectangleOp(2,6); r2=RectangleOp(3,4)
print('Same area?', r1==r2)
```

Output:

```
Same area? True
```